

Developer Slides

TGIR QuantLib Tools

TG Investments and Research

2026-04-06

Repo Structure

- `portfolio.py`: pricing and analytics.
- `tgir_quantlib_tools/`: Flask app, routes, state, and model context.
- `templates/`: UI layer.
- `tests/`: smoke, calibration, and identity tests.
- `docs/latex/`: audience documentation set.

Application Flow

- `create_app()` builds the Flask app.
- Routes load and save normalized session state.
- Pricing calls stay centralized in `portfolio.py`.
- Templates should render prepared context, not calculate prices.

Extension Rules

- Add new state defaults and normalization first.
- Extend `TRADE_SEQUENCE`, titles, pricing, forms, and templates together.
- Keep the Flask app thin.
- Add tests before considering the change complete.

Developer Commands

```
./venv/bin/python app.py  
./venv/bin/python -m unittest discover -s tests  
./venv/bin/python -m py_compile portfolio.py tgir_quantlib_tools/*.py  
make -C docs/latex
```

Review Checklist

- Pricing logic centralized in `portfolio.py`.
- State normalized before use.
- Calibration still reprices.
- Route smoke tests pass.
- Docs updated for the right audience.